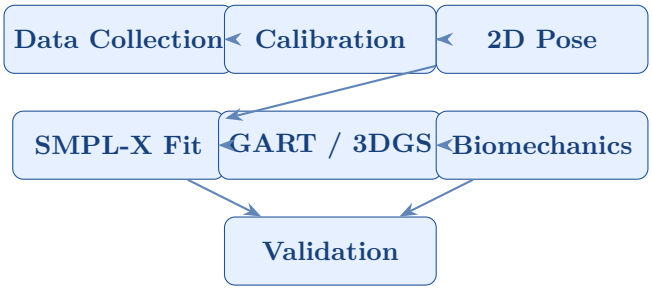
Part 4

다시점 촬영 데이터 분석 파이프라인

데이터 수집부터 3DGS 인체 재구성, 생체역학 지표 산출까지



컴퓨터 공학 × 스포츠의학

2026

본 교재는 Part 1 (NeRF→GART), Part 2 (3DGS+동작분석), Part 3 (투구 실험설계)의 후속편으로,
촬영된 데이터를 실제로 처리·분석하는 전체 파이프라인을 단계별로 다룹니다.

Contents

서문 **3**

1 데이터 수집과 전처리 **4**
1.1 다시점 영상 데이터 구조 4
1.1.1 네이밍 컨벤션 4
1.1.2 메타데이터 파일 5
1.2 프레임 동기화 검증 5
1.2.1 동기화 방법 5
1.2.2 소프트웨어 기반 동기화 검증 5
1.3 영상 전처리 6
1.3.1 디베이어링(Debayering) 6
1.3.2 렌즈 왜곡 보정 6
1.3.3 색 보정과 화이트밸런스 6
1.4 Vicon 마커 데이터 전처리 6
1.4.1 갭 필링(Gap Filling) 6
1.4.2 라벨링 7
1.4.3 필터링 7

2 카메라 보정과 COLMAP **8**
2.1 개별 카메라 내부 보정 8
2.1.1 체커보드 보정 8
2.1.2 ChArUco 보드 9
2.2 COLMAP SfM/MVS 파이프라인 9
2.2.1 Bundle Adjustment 11
2.3 다시점 외부 보정 11
2.3.1 동시 보정 vs 순차 보정 11
2.3.2 알려진 내부 파라미터 활용 11
2.4 Vicon–RGB 좌표계 정합 11
2.5 보정 품질 평가 12
2.5.1 재투영 오차 12
2.5.2 삼각측량 정밀도 12

3 2D 자세 추정과 다시점 삼각측량 **13**
3.1 프레임별 2D 자세 추정 13
3.2 고속 영상의 특수 문제 13
3.2.1 모션 블러 13
3.2.2 반복 질감(Repetitive Texture) 14
3.3 다시점 삼각측량 14
3.3.1 DLT(Direct Linear Transform) 14

3.3.2 RANSAC 기반 이상치 제거 . . . 14
3.4 시계열 필터링 . . . 15
3.4.1 Butterworth 저역통과 필터 . . . 15
3.4.2 Savitzky–Golay 필터 . . . 15

4 다시점 SMPL-X 피팅 17
4.1 EasyMocap 파이프라인 상세 . . . 17
4.2 다시점 목적 함수와 최적화 . . . 17
4.2.1 2D 재투영 손실 . . . 17
4.2.2 3D 관절 손실 . . . 17
4.2.3 사전 확률 정규화 . . . 18
4.3 시간적 일관성 정규화 . . . 18
4.4 피팅 결과 시각화 및 디버깅 . . . 18
4.4.1 체크리스트 . . . 18
4.4.2 흔한 피팅 실패와 해결 . . . 19

5 3DGS 인체 재구성 20
5.1 정적 배경 3DGS 학습 . . . 20
5.2 동적 인체: GART 학습 파이프라인 . . . 20
5.2.1 LBS(Linear Blend Skinning)를 통한 변환 . . . 21
5.2.2 학습 가능 파라미터 . . . 21
5.3 렌더링 기반 자세 정제 . . . 21
5.4 학습 모니터링 . . . 22
5.4.1 정량적 지표 . . . 22
5.4.2 학습 곡선 해석 . . . 22
5.5 디버깅 가이드 . . . 22

6 생체역학 지표 산출 23
6.1 SMPL에서 해부학적 관절 각도로 . . . 23
6.1.1 ISB 분해 . . . 23
6.2 가상 마커 추출과 OpenSim IK . . . 24
6.3 역동역학(Inverse Dynamics) . . . 24
6.4 투구 이벤트 자동 검출 . . . 25
6.5 핵심 지표 산출 . . . 25
6.5.1 외반 토크 (Valgus Torque) . . . 25
6.5.2 최대 외회전 각도 . . . 26
6.5.3 팔 속도 . . . 26

7 Vicon 검증과 오차 분석 27
7.1 시간 정합 확인 . . . 27
7.2 자세 정확도 평가 . . . 27
7.2.1 MPJPE (Mean Per Joint Position Error) . . . 27
7.2.2 관절각 RMSE . . . 28
7.3 운동역학 정확도 . . . 28
7.4 Bland–Altman 분석과 ICC . . . 28
7.4.1 Bland–Altman 분석 . . . 28
7.4.2 ICC (Intraclass Correlation Coefficient) . . . 29
7.5 오차 원인 분석 . . . 29

8 시각화와 보고서 생성 **31**
8.1 3DGS 렌더링 + 생체역학 오버레이 31
8.1.1 관절 토크 컬러맵 31
8.1.2 벡터 필드 시각화 31
8.2 시계열 관절 각도/토크 플롯 32
8.3 투구별 비교 대시보드 32
8.4 자동 보고서 생성 파이프라인 32

9 수학적 부록 **34**
9.1 Procrustes 정합 34
9.1.1 문제 정의 34
9.1.2 풀이 34
9.2 Butterworth 필터 설계 34
9.2.1 아날로그 프로토타입 34
9.2.2 디지털 변환 35
9.2.3 양방향 필터링(Zero-Phase) 35
9.3 Bland–Altman 분석의 통계적 기초 35
9.3.1 가정 35
9.3.2 일치한계(Limits of Agreement) 35
9.3.3 비례 바이어스 검정 35
9.4 ICC 계산 36
9.4.1 Two-Way Mixed Model, Absolute Agreement 36
9.4.2 신뢰구간 36

용어집 **38**

서문

Part 3까지는 "무엇을 촬영할 것인가"와 "어떤 장비를 어떻게 배치할 것인가"에 집중했다. 실험 설계가 끝나면 자연스럽게 떠오르는 질문은 이것이다: **"촬영이 끝난 뒤, 이 수백 GB의 영상과 모션캡처 파일을 어떻게 처리하여 의미 있는 생체역학 지표로 바꿀 것인가?"**

Part 4는 이 질문에 대한 답이다. 7대의 카메라에서 나온 동기화 영상과 Vicon 마커 데이터를 입력으로 받아,

1. 카메라 보정과 좌표계 정합,
2. 2D/3D 자세 추정,
3. SMPL-X 파라메트릭 인체 모델 피팅,
4. 3D Gaussian Splatting(GART) 기반 포토리얼리스틱 인체 재구성,
5. 관절 각도·토크·힘 등 생체역학 지표 산출,
6. Vicon gold-standard와의 정량적 검증

까지 이르는 **end-to-end 데이터 분석 파이프라인**을 단계별로 설명한다.

각 챕터는 이론적 배경을 먼저 제시한 뒤, 실제로 사용하는 명령어와 코드 스니펫을 함께 보여주어 독자가 직접 재현할 수 있도록 구성하였다. 특히 고속 투구 동작이라는 난이도 높은 시나리오를 기준으로 설명하되, 일반적인 보행 분석이나 다른 스포츠 동작에도 동일한 파이프라인을 적용할 수 있음을 강조한다.

Important

본 교재의 코드 예시는 Python 3.10+, PyTorch 2.x, CUDA 11.8+ 환경을 기준으로 한다. 운영체제는 Ubuntu 22.04를 권장하며, macOS에서도 대부분의 전처리 단계를 수행할 수 있다.

Chapter 1

데이터 수집과 전처리

1.1 다시점 영상 데이터 구조

촬영이 완료되면 가장 먼저 해야 할 일은 데이터를 체계적으로 정리하는 것이다. 일관된 디렉토리 구조는 이후 모든 자동화 스크립트의 전제 조건이 된다.

권장 디렉토리 레이아웃

```
project/
  subject_01/
    set_001/
      device1_synced.MOV     # Camera 1
      device2_synced.MOV     # Camera 2
      ...
      device7_synced.MOV     # Camera 7
      _synced_meta.json      # Sync metadata
      sync_offsets.json      # Per-device time offsets
    set_002/
      ...
  calibration/
    intrinsics/              # Per-camera intrinsic params
    extrinsics/              # Multi-camera extrinsic params
    charuco_images/          # Calibration board images
```

Key Idea

파일명에 `device{N}_synced` 규칙을 사용하면 카메라 인덱스를 파싱하기 쉽고, `_synced` 접미사로 동기화 완료 여부를 즉시 확인할 수 있다.

1.1.1 네이밍 컨벤션

다시점 촬영 시스템에서는 수십~수백 개의 파일이 생성되므로, 명확한 네이밍 규칙이 필수적이다.

- **피험자 ID**: `subject_XX` (2자리 정수)
- **세트 번호**: `set_XXX` (3자리 정수, 시행 순서)
- **카메라 ID**: `deviceN` ($N = 1, \ldots, 7$)
- **동기화 상태**: `_synced` (동기화 완료) / `_raw` (원본)

1.1.2 메타데이터 파일

`_synced_meta.json`에는 각 카메라 영상의 원본 경로, 프레임 레이트, 해상도, 녹화 시작·종료 타임스탬프가 기록된다. `sync_offsets.json`에는 각 디바이스 간의 시간 오프셋(밀리초 단위)이 저장된다.

sync_offsets.json 예시

```
{
  "device1": 0.0,       # Reference camera
  "device2": -3.2,      # 3.2ms earlier
  "device3": 1.5,       # 1.5ms later
  ...
}
```

1.2 프레임 동기화 검증

다시점 촬영에서 가장 중요한 전처리 단계는 **시간 동기화(temporal synchronization)**이다. 카메라 간 프레임이 정확히 같은 시점을 가리키지 않으면, 삼각측량 결과에 체계적 오차가 발생한다.

1.2.1 동기화 방법

세 가지 대표적 동기화 방법을 비교한다.

Table 1.1: 다시점 동기화 방법 비교

방법	원리	정밀도	난이도
하드웨어 젠록	공통 클럭 신호로 셔터 동기	$< 0.1\,\mathrm{ms}$	높음
LED 플래시	촬영 시작 시 LED 점멸, 프레임에서 검출	$\sim$1 프레임	낮음
오디오 싱크	클랩보드/비프음의 파형 상관 분석	$\sim 1\,\mathrm{ms}$	중간

1.2.2 소프트웨어 기반 동기화 검증

하드웨어 동기화를 사용하더라도 사후 검증이 필수적이다. 대표적 방법은 빠르게 움직이는 객체(예: 투구 중 공)의 위치를 각 카메라에서 추적한 뒤, 에피폴라 제약(epipolar constraint)을 만족하는지 확인하는 것이다.

Definition 1.1 (에피폴라 제약)**.** 두 카메라 i, j의 기본 행렬(fundamental matrix)을 $\mathbf{F}_{ij}$라 하면, 대응점 $\mathbf{x}_i, \mathbf{x}_j$에 대해

$$\mathbf{x}_j^\top \mathbf{F}_{ij}\, \mathbf{x}_i = 0 \tag{1.1}$$

이 성립해야 한다. 동기화 오차가 존재하면 이 값이 0에서 벗어난다.

Remark 1.1. 240 Hz 촬영에서 1 프레임의 동기화 오차는 약 4.17 ms에 해당한다. 투구 중 손 끝의 속도가 $\sim$30 m/s라면, 1 프레임 오차는 약 12.5 cm의 위치 오차를 야기한다. 따라서 고속 동작에서는 서브프레임 정밀도의 동기화가 필수적이다.

1.3 영상 전처리

1.3.1 디베이어링(Debayering)

RAW 영상을 촬영한 경우, Bayer 패턴에서 풀 컬러 이미지로의 변환(디베이어링)이 필요하다. 일반적인 소비자 카메라(MOV/MP4)의 경우 카메라 내부에서 이미 처리되어 이 단계를 건너뛸 수 있다.

1.3.2 렌즈 왜곡 보정

광각 렌즈를 사용하면 방사형(radial) 및 접선형(tangential) 왜곡이 발생한다. OpenCV의 왜곡 모델은 다음과 같다:

$$\begin{aligned} x_{\text{dist}} &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{dist}} &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{aligned} \tag{1.2}$$

여기서 $r^2 = x^2 + y^2$, (k_1, k_2, k_3)는 방사형 왜곡 계수, (p_1, p_2)는 접선형 왜곡 계수이다.

OpenCV 왜곡 보정

```python
import cv2
import numpy as np

# Load calibration
K = np.load("intrinsics/cam1_K.npy")      # 3x3
dist = np.load("intrinsics/cam1_dist.npy")  # [k1,k2,p1,p2,k3]

# Undistort
map1, map2 = cv2.initUndistortRectifyMap(
    K, dist, None, K, (W, H), cv2.CV_32FC1
)
undistorted = cv2.remap(frame, map1, map2, cv2.INTER_LINEAR)
```

1.3.3 색 보정과 화이트밸런스

다시점 시스템에서 모든 카메라의 색 응답이 동일해야 뷰 간 일관된 학습이 가능하다. 촬영 전에 ColorChecker 차트를 촬영하여 카메라별 색 변환 행렬 $\mathbf{C}_i \in \mathbb{R}^{3\times3}$을 추정한다:

$$\mathbf{I}_{\text{corrected}} = \mathbf{C}_i \, \mathbf{I}_{\text{raw}} \tag{1.3}$$

1.4 Vicon 마커 데이터 전처리

1.4.1 갭 필링(Gap Filling)

마커가 일시적으로 가려지면 궤적에 빈 구간(gap)이 생 다. Vicon Nexus에서는 다음 방법을 제공한다:

- **Spline Fill**: 3차 스플라인 보간 — 짧은 갭(<10 프레임)에 적합
- **Pattern Fill**: 인접 마커의 움직임 패턴을 참조 — 긴 갭에 유용
- **Rigid Body Fill**: 강체 가정 하에 다른 마커로부터 복원

1.4.2 라벨링

자동 라벨링 후 반드시 수동 검증이 필요하다. 특히 고속 동작에서는 마커 교차(swap)가 빈번하게 발생하므로, 스켈레톤 모델을 오버레이하여 프레임별로 확인한다.

1.4.3 필터링

마커 궤적에 포함된 고주파 노이즈를 제거하기 위해 저역통과 필터를 적용한다. 투구 동작에서의 일반적 차단 주파수는 12–25 Hz이다 (잔차 분석 기반 선택).

Biomechanics

잔차 분석(Residual Analysis) [1]: 다양한 차단 주파수 f_c에 대해 필터링된 신호와 원본의 잔차 RMS를 계산한다. 잔차가 직선적으로 증가하기 시작하는 지점이 최적 f_c이다. 이는 신호와 노이즈의 파워 스펙트럼이 분리되는 주파수에 해당한다.

Chapter 2

카메라 보정과 COLMAP

2.1 개별 카메라 내부 보정

카메라 내부 파라미터(intrinsic parameters)는 3D 점을 2D 이미지 좌표로 매핑하는 투영 모델의 핵심이다.

Definition 2.1 (카메라 내부 행렬)**.** 핀홀 카메라의 내부 행렬은 다음과 같다:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \tag{2.1}$$

여기서 (f_x, f_y)는 초점 거리(픽셀 단위), (c_x, c_y)는 주점(principal point)이다.

2.1.1 체커보드 보정

Zhang의 방법 [2]을 사용하여 체커보드 패턴으로 내부 파라미터와 왜곡 계수를 동시에 추정한다.

체커보드 보정 코드

```python
import cv2
import glob

BOARD = (9, 6)  # inner corners
objp = np.zeros((BOARD[0]*BOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:BOARD[0], 0:BOARD[1]].T.reshape(-1, 2)
objp *= 30.0  # 30mm square size

obj_pts, img_pts = [], []
for path in sorted(glob.glob("calib_images/cam1_*.png")):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    ret, corners = cv2.findChessboardCorners(img, BOARD)
    if ret:
        corners = cv2.cornerSubPix(img, corners, (11,11), (-1,-1),
            (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_COUNT, 30, 0.001)
            )
        obj_pts.append(objp)
        img_pts.append(corners)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
    obj_pts, img_pts, img.shape[::-1], None, None
)
print(f"RMS reprojection error: {ret:.4f} px")
```

Important

재투영 오차(RMS)가 0.5 픽셀을 초과하면 보정 이미지를 재검토해야 한다. 체커보드가 다양한 각도와 위치에서 촬영되었는지, 이미지 전체 영역을 커버하는지 확인한다.

2.1.2 ChArUco 보드

ChArUco 보드는 체커보드와 ArUco 마커를 결합한 것으로, 부분적으로 가려져도 코너를 검출할 수 있어 다시점 환경에서 특히 유용하다.

2.2 COLMAP SfM/MVS 파이프라인

COLMAP은 Structure from Motion(SfM)과 Multi-View Stereo(MVS)를 수행하는 표준 도구로, 3D Gaussian Splatting의 초기 점군 생성에 핵심적으로 사용된다.

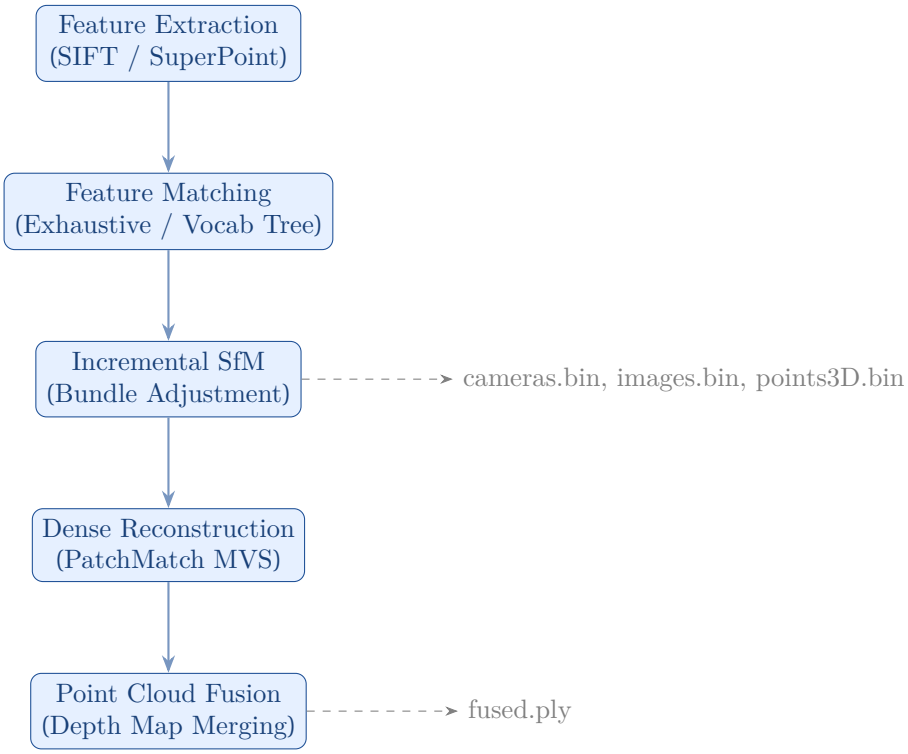


Figure 2.1: COLMAP 파이프라인 흐름도

COLMAP CLI 사용법

```
# 1. Feature extraction
colmap feature_extractor \
    --database_path db.db \
    --image_path images/ \
    --ImageReader.camera_model OPENCV \
    --ImageReader.single_camera_per_folder 1

# 2. Feature matching
colmap exhaustive_matcher --database_path db.db

# 3. Sparse reconstruction (SfM)
mkdir sparse
colmap mapper \
    --database_path db.db \
    --image_path images/ \
    --output_path sparse/

# 4. Dense reconstruction (optional for 3DGS)
colmap image_undistorter \
    --image_path images/ \
    --input_path sparse/0 \
    --output_path dense/
```

2.2.1 Bundle Adjustment

Bundle Adjustment(BA)는 모든 카메라 자세와 3D 점 위치를 동시에 최적화하는 비선형 최소제곱 문제이다:

$$\min_{\{\mathbf{R}_i,\mathbf{t}_i\},\{\mathbf{p}_j\}} \sum_{i,j} \rho\Big(\|\pi(\mathbf{R}_i,\mathbf{t}_i,\mathbf{K}_i,\mathbf{p}_j)-\mathbf{x}_{ij}\|^2\Big) \tag{2.2}$$

여기서 π는 투영 함수, ρ는 로버스트 손실 함수(Cauchy/Huber), $\mathbf{x}_{ij}$는 카메라 i에서의 점 j의 관측이다.

2.3 다시점 외부 보정

2.3.1 동시 보정 vs 순차 보정

Table 2.1: 외부 보정 전략 비교

전략	장점	단점
동시(Simultaneous)	모든 카메라 쌍의 관계를 한 번에 최적화, 전역 일관성 보장	대규모 시스템에서 계산 비용 높음
순차(Sequential)	카메라를 하나씩 추가하며 점진적 보정, 구현 용이	오차 누적(drift) 가능

2.3.2 알려진 내부 파라미터 활용

사전에 보정된 내부 파라미터를 COLMAP에 고정(fix)하면 외부 보정의 정확도가 향상된다:

내부 파라미터 고정

```
colmap mapper \
    --Mapper.ba_refine_focal_length 0 \
    --Mapper.ba_refine_principal_point 0 \
    --Mapper.ba_refine_extra_params 0
```

2.4 Vicon–RGB 좌표계 정합

Vicon 시스템과 카메라 시스템은 서로 다른 좌표계를 사용하므로, 두 좌표계 간의 강체 변환(rigid transformation)을 추정해야 한다.

Definition 2.2 (Procrustes 정합). N개의 대응점 $\{\mathbf{a}_i\} \subset \mathbb{R}^3$ (Vicon)과 $\{\mathbf{b}_i\} \subset \mathbb{R}^3$ (카메라)이 주어졌을 때, 최적의 회전 $\mathbf{R}^*$, 이동 $\mathbf{t}^*$, 스케일 s^*를 구한다:

$$\mathbf{R}^*,\mathbf{t}^*,s^* = \arg\min_{\mathbf{R},\mathbf{t},s} \sum_{i=1}^{N} \|\mathbf{b}_i-(s\mathbf{R}\mathbf{a}_i+\mathbf{t})\|^2 \tag{2.3}$$

실용적으로, Vicon 마커 중 최소 4개를 RGB 이미지에서도 식별 가능한 위치에 부착하여 공통 랜드마크로 활용한다. 반사 마커는 적외선에서만 밝게 보이므로, 별도의 컬러 마커나 체커보드 코너를 공통점으로 사용하는 것이 일반적이다.

2.5 보정 품질 평가

2.5.1 재투영 오차

카메라 보정의 가장 기본적인 품질 지표는 재투영 오차이다:

$$e_{\text{reproj}} = \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} \|\pi(\mathbf{R}_i, \mathbf{t}_i, \mathbf{K}_i, \mathbf{p}_j) - \mathbf{x}_{ij}\|_2 \tag{2.4}$$

Table 2.2: 재투영 오차 기준

수준	재투영 오차 (px)
우수	< 0.3
양호	$0.3 \sim 0.5$
주의 필요	$0.5 \sim 1.0$
재보정 필요	> 1.0

2.5.2 삼각측량 정밀도

두 카메라 이상에서 관측된 점의 3D 위치를 삼각측량으로 복원한 뒤, 복원 정밀도를 평가한다. 특히 기저선(baseline) 대비 깊이 비율이 중요하다:

$$\sigma_z \propto \frac{z^2}{f \cdot B} \tag{2.5}$$

여기서 z는 깊이, f는 초점 거리, B는 기저선 길이이다. 기저선이 짧으면 깊이 추정의 불확실성이 급격히 증가한다.

Chapter 3

2D 자세 추정과 다시점 삼각측량

3.1 프레임별 2D 자세 추정

각 카메라 영상의 모든 프레임에서 2D 관절 위치를 추정한다. 현재 최고 성능의 모델은 ViTPose [3]와 RTMPose [4]이다.

Computer Vision

ViTPose는 Vision Transformer 기반 자세 추정 모델로, 사전학습된 MAE(Masked Autoencoder) 백본 위에 간단한 디코더를 올린 구조이다. COCO 벤치마크에서 AP 75.8 (ViTPose-H)을 달성하며, 큰 모델 크기 대비 높은 정확도를 제공한다.
RTMPose는 실시간 추론에 특화된 경량 모델로, SimCC(Simplified Coordinate Classification) 기반 좌표 예측을 사용한다. 240fps 영상의 실시간 처리가 가능하다.

MMPose를 이용한 배치 추론

```python
from mmpose.apis import MMPoseInferencer

inferencer = MMPoseInferencer(
    pose2d='td-hm_ViTPose-huge_8xb64-210e_coco-256x192'
)

# Process all frames from one camera
results = inferencer(
    'frames/cam1/',
    show=False,
    out_dir='pose2d/cam1/',
    return_datasamples=True
)
```

3.2 고속 영상의 특수 문제

3.2.1 모션 블러

240 Hz 이상에서도 투구 릴리스 순간의 손·공은 심각한 모션 블러를 보일 수 있다. 셔터 속도를 1/1000 s 이하로 설정하여 블러를 최소화해야 한다.

Important

셔터 속도를 높이면 조리개를 더 열거나 ISO를 높여야 하므로 피사계 심도(DoF)가 얕아지거나 노이즈가 증가한다. 240 Hz + 1/1000 s 조합에서는 충분한 조명(>2000 lux)이 필수적이다.

3.2.2 반복 질감(Repetitive Texture)

실내 체육관의 균일한 바닥이나 단색 벽면은 특징점 매칭을 어렵게 만든다. 해결 방법:

- 바닥에 비반복적 패턴(예: 랜덤 도트) 부착
- SuperPoint/SuperGlue 같은 학습 기반 특징점 사용
- 매칭 시 에피폴라 제약을 적극 활용하여 잘못된 매칭 제거

3.3 다시점 삼각측량

C개의 카메라에서 관측된 2D 관절 위치 $\{\mathbf{x}_c\}_{c=1}^{C}$로부터 3D 위치 $\mathbf{p} \in \mathbb{R}^3$를 복원한다.

3.3.1 DLT(Direct Linear Transform)

각 카메라의 투영 행렬을 $\mathbf{P}_c = \mathbf{K}_c[\mathbf{R}_c|\mathbf{t}_c] \in \mathbb{R}^{3\times 4}$라 하면, 관측 $\mathbf{x}_c = (u_c, v_c)$에 대해 다음 선형 방정식을 세울 수 있다:

$$\begin{pmatrix} u_c\,\mathbf{p}_c^{3\top} - \mathbf{p}_c^{1\top} \\ v_c\,\mathbf{p}_c^{3\top} - \mathbf{p}_c^{2\top} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix} = \mathbf{0} \tag{3.1}$$

여기서 $\mathbf{p}_c^{k\top}$는 $\mathbf{P}_c$의 k번째 행이다. 모든 카메라의 방정식을 쌓아 $\mathbf{A}\mathbf{p}_h = \mathbf{0}$ 형태로 만들고, SVD로 풀면 최소제곱 해를 얻는다.

DLT 삼각측량 구현

```python
def triangulate_dlt(proj_matrices, points_2d):
    """
    proj_matrices: list of (3,4) projection matrices
    points_2d: list of (2,) 2D observations
    returns: (3,) 3D point
    """
    n = len(proj_matrices)
    A = np.zeros((2*n, 4))
    for i, (P, x) in enumerate(zip(proj_matrices, points_2d)):
        A[2*i]   = x[0]*P[2] - P[0]
        A[2*i+1] = x[1]*P[2] - P[1]
    _, _, Vt = np.linalg.svd(A)
    X = Vt[-1]
    return X[:3] / X[3]
```

3.3.2 RANSAC 기반 이상치 제거

일부 카메라에서 2D 검출이 실패하거나 부정확할 수 있다. RANSAC을 적용하여 이상치 뷰를 제거한다:

Algorithm 1 RANSAC 삼각측량

Require: 카메라 투영 행렬 $\{\mathbf{P}_c\}$, 2D 관측 $\{\mathbf{x}_c\}$, 임계값 τ

```
for k = 1 to K_max do
    2개의 카메라를 랜덤 선택
    DLT로 3D 점 p 추정
    모든 카메라에 재투영: x̂_c = π(P_c, p)
    인라이어 집합: I = {c : ‖x̂_c − x_c‖ < τ}
    if |I| > 최대 인라이어 수 then
        최적 해 갱신
    end if
end for
최적 인라이어 집합으로 DLT 재수행
return p*
```

3.4 시계열 필터링

삼각측량으로 얻은 3D 관절 궤적에는 검출 노이즈로 인한 고주파 변동이 포함된다. 이를 제거하기 위해 시계열 필터를 적용한다.

3.4.1 Butterworth 저역통과 필터

n차 Butterworth 필터의 주파수 응답은:

$$|H(f)|^2 = \frac{1}{1 + (f/f_c)^{2n}} \tag{3.2}$$

여기서 f_c는 차단 주파수, n은 필터 차수이다. 위상 지연을 제거하기 위해 **양방향 필터링(zero-phase filtering)**을 적용한다.

양방향 Butterworth 필터

```python
from scipy.signal import butter, filtfilt

def butterworth_filter(data, cutoff, fs, order=4):
    nyq = 0.5 * fs
    b, a = butter(order, cutoff / nyq, btype='low')
    return filtfilt(b, a, data, axis=0)

# 240 Hz data, 15 Hz cutoff
joints_filtered = butterworth_filter(joints_3d, 15, 240)
```

3.4.2 Savitzky–Golay 필터

다항식 피팅 기반의 Savitzky–Golay 필터는 신호의 피크를 잘 보존하는 장점이 있다:

$$\hat{y}_i = \sum_{m=-M}^{M} c_m \, y_{i+m} \tag{3.3}$$

여기서 c_m은 $2M+1$ 크기 윈도우 내에서 p차 다항식 피팅으로 결정되는 계수이다.

Key Idea

Butterworth는 주파수 영역에서, Savitzky–Golay는 시간 영역에서 동작한다. 투구처럼 충격적(impulsive) 신호가 포함된 경우 Savitzky–Golay가 피크 보존에 유리하고, 주기적 보행 동작에서는 Butterworth가 더 깔끔한 결과를 준다.

Chapter 4

다시점 SMPL-X 피팅

4.1 EasyMocap 파이프라인 상세

EasyMocap [5]은 다시점 영상에서 SMPL/SMPL-X 파라미터를 추정하는 오픈소스 프레임워크이다. 입력으로 카메라 보정 파라미터와 2D 자세 검출 결과를 받아, 최적화 기반으로 인체 모델을 피팅한다.

EasyMocap 처리 흐름

1. 카메라 파라미터 로드 (내부 + 외부)
2. 프레임별 2D 관절 검출 결과 로드
3. 다시점 삼각측량으로 초기 3D 관절 추정
4. SMPL-X 모델 초기화 (평균 체형 + 중립 자세)
5. 단계적 최적화: 전역 자세 → 체형 → 관절 자세 → 손/얼굴
6. 결과 저장: 프레임별 SMPL-X 파라미터 $(\boldsymbol{\beta}, \boldsymbol{\theta}, \mathbf{t})$

4.2 다시점 목적 함수와 최적화

SMPL-X 피팅의 핵심은 다시점 2D 관측과 3D 모델을 일치시키는 목적 함수의 설계이다.

$$\mathcal{L}_{\text{total}} = \lambda_{\text{2D}}\mathcal{L}_{\text{2D}} + \lambda_{\text{3D}}\mathcal{L}_{\text{3D}} + \lambda_{\beta}\mathcal{L}_{\beta} + \lambda_{\theta}\mathcal{L}_{\theta} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}} \tag{4.1}$$

4.2.1 2D 재투영 손실

C개 카메라의 2D 관측과 SMPL-X 관절의 투영을 비교한다:

$$\mathcal{L}_{\text{2D}} = \sum_{c=1}^{C}\sum_{j=1}^{J} w_{cj} \left\| \pi_c\big(J_j(\boldsymbol{\beta}, \boldsymbol{\theta})\big) - \hat{\mathbf{x}}_{cj} \right\|^2 \tag{4.2}$$

여기서 $J_j(\boldsymbol{\beta}, \boldsymbol{\theta})$는 SMPL-X의 j번째 관절의 3D 위치, $\hat{\mathbf{x}}_{cj}$는 카메라 c에서의 2D 검출, w_{cj}는 검출 신뢰도이다.

4.2.2 3D 관절 손실

삼각측량으로 얻은 3D 관절과의 차이를 최소화한다:

$$\mathcal{L}_{\text{3D}} = \sum_{j=1}^{J} \left\| J_j(\boldsymbol{\beta}, \boldsymbol{\theta}) - \hat{\mathbf{p}}_j \right\|^2 \tag{4.3}$$

4.2.3 사전 확률 정규화

체형과 자세의 과도한 변형을 방지한다:

$$\mathcal{L}_\beta = \|\boldsymbol{\beta}\|^2, \qquad \mathcal{L}_\theta = \|\boldsymbol{\theta}\|^2 \tag{4.4}$$

Important

$\mathcal{L}_\theta$에 단순 L_2 정규화를 사용하면 모든 관절이 중립 자세(T-pose)로 당겨진다. 실제로는 VPoser [6] 같은 학습된 자세 사전(prior)을 사용하여 해부학적으로 타당한 자세 공간 내에서 최적화하는 것이 바람직하다.

4.3 시간적 일관성 정규화

프레임별 독립 피팅은 시간적 떨림(jitter)을 유발한다. 인접 프레임 간의 파라미터 변화를 제한하는 정규화를 추가한다:

$$\mathcal{L}_{\text{smooth}} = \sum_{t=2}^{T} \left(\|\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}\|^2 + \gamma \|\boldsymbol{\theta}_t - 2\boldsymbol{\theta}_{t-1} + \boldsymbol{\theta}_{t-2}\|^2 \right) \tag{4.5}$$

첫째 항은 속도 정규화(1차 유한차분), 둘째 항은 가속도 정규화(2차 유한차분)이다. γ는 일반적으로 0.5–2.0 사이로 설정한다.

Key Idea

가속도 정규화는 등속 운동은 허용하면서 급격한 방향 전환만 억제한다. 투구의 가속 구간(arm cocking)에서는 γ를 낮추고, 팔로-스루에서는 높이는 적응형 전략이 효과적이다.

4.4 피팅 결과 시각화 및 디버깅

4.4.1 체크리스트

피팅 결과를 검증하기 위한 체계적 체크리스트:

1. **실루엣 오버레이**: 각 카메라 뷰에서 SMPL-X 메시를 원본 이미지에 오버레이
2. **관절 재투영**: 3D 관절을 2D로 투영하여 검출 결과와 비교
3. **체형 일관성**: 모든 프레임에서 $\boldsymbol{\beta}$가 일정한지 확인 (같은 피험자이므로)
4. **관절 범위**: 해부학적으로 불가능한 관절 각도가 없는지 확인
5. **침투 검사**: 자기 교차(self-intersection)가 없는지 확인

피팅 결과 시각화

```python
import trimesh
import pyrender

# Load SMPL-X result
body_model = smplx.create(model_path, model_type='smplx')
output = body_model(betas=betas, body_pose=pose,
                    global_orient=orient, transl=transl)
vertices = output.vertices.detach().numpy()[0]
faces = body_model.faces

# Render overlay
mesh = trimesh.Trimesh(vertices, faces)
scene = pyrender.Scene()
scene.add(pyrender.Mesh.from_trimesh(mesh))
# ... add camera, render, blend with original image
```

4.4.2 흔한 피팅 실패와 해결

Table 4.1: SMPL-X 피팅 실패 유형과 해결 방법

증상	원인	해결
메시가 뒤집어짐	전역 회전 초기화 실패	초기 자세를 수동으로 설정
팔다리 관통	자기 교차 정규화 부재	$\mathcal{L}_{\text{penetration}}$ 추가
떨림(jitter)	시간 정규화 가중치 부족	λ_{smooth} 증가
체형 왜곡	뷰 수 부족	카메라 추가 또는 λ_β 증가

Chapter 5

3DGS 인체 재구성

5.1 정적 배경 3DGS 학습

동적 인체를 재구성하기 전에, 먼저 빈 장면(인체 없는 상태)의 정적 3DGS를 학습한다. 이 배경 모델은 이후 동적 인체 분리(foreground/background decomposition)에 활용된다.

배경 3DGS 학습 단계

1. 인체가 없는 상태에서 다시점 촬영 (또는 인체 마스크로 배경만 추출)
2. COLMAP으로 초기 점군 생성
3. 3DGS 학습: 30,000 iterations 기준 약 20–30분 (RTX 4090)
4. 렌더링하여 배경 재구성 품질 확인

3DGS의 각 Gaussian g_k는 다음 파라미터로 정의된다:

$$g_k = \{\boldsymbol{\mu}_k,\ \boldsymbol{\Sigma}_k,\ \alpha_k,\ \mathbf{c}_k\} \tag{5.1}$$

여기서 $\boldsymbol{\mu}_k \in \mathbb{R}^3$은 중심, $\boldsymbol{\Sigma}_k \in \mathbb{R}^{3\times 3}$은 공분산(스케일+회전으로 분해), $\alpha_k \in [0,1]$은 불투명도, $\mathbf{c}_k$는 SH 계수(색상)이다.

5.2 동적 인체: GART 학습 파이프라인

GART(Gaussian Articulated Representation of Transformable humans)는 SMPL-X의 스키닝 가중치를 활용하여 정규 공간(canonical space)의 Gaussian을 각 프레임의 자세로 변환하는 구조이다.

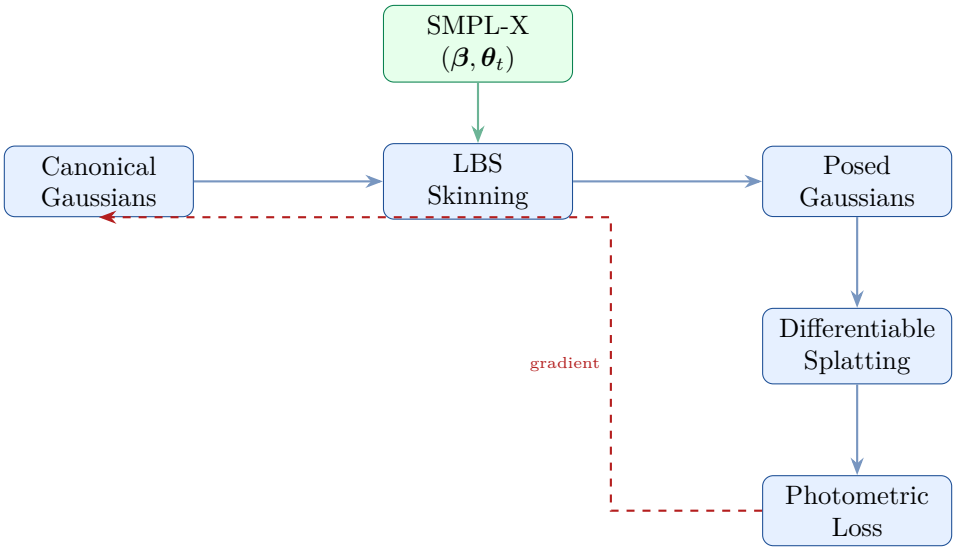


Figure 5.1: GART 학습 루프: 정규 공간 Gaussian을 SMPL-X 자세로 변환 후 렌더링하여 학습

5.2.1 LBS(Linear Blend Skinning)를 통한 변환

정규 공간의 Gaussian 중심 $\boldsymbol{\mu}_k^{\text{can}}$을 자세 $\boldsymbol{\theta}_t$로 변환한다:

$$\boldsymbol{\mu}_k^{\text{posed}} = \left(\sum_{b=1}^{B} w_{kb} \, \mathbf{T}_b(\boldsymbol{\theta}_t) \right) \boldsymbol{\mu}_k^{\text{can}} \tag{5.2}$$

여기서 w_{kb}는 Gaussian k의 본(bone) b에 대한 스키닝 가중치, $\mathbf{T}_b(\boldsymbol{\theta}_t) \in SE(3)$는 본 b의 변환 행렬이다.

공분산도 함께 변환된다:

$$\boldsymbol{\Sigma}_k^{\text{posed}} = \mathbf{R}_{\text{lbs}} \, \boldsymbol{\Sigma}_k^{\text{can}} \, \mathbf{R}_{\text{lbs}}^{\top} \tag{5.3}$$

여기서 $\mathbf{R}_{\text{lbs}}$는 LBS 변환의 회전 성분이다.

5.2.2 학습 가능 파라미터

GART에서 최적화하는 파라미터는:

- 정규 공간 Gaussian 속성: $\{\boldsymbol{\mu}_k^{\text{can}}, \boldsymbol{\Sigma}_k^{\text{can}}, \alpha_k, \mathbf{c}_k\}$
- 스키닝 가중치 오프셋: Δw_{kb} (SMPL-X 기본 가중치에서의 미세 조정)
- (선택) 비강체 변형 네트워크: $\Delta \boldsymbol{\mu}_k = f_\phi(\boldsymbol{\mu}_k^{\text{can}}, \boldsymbol{\theta}_t)$

5.3 렌더링 기반 자세 정제

SMPL-X 피팅 결과에 남아 있는 자세 오차를 렌더링 손실로 정제할 수 있다. 이는 분석-합성(analysis-by-synthesis) 접근법이다:

$$\boldsymbol{\theta}_t^* = \arg\min_{\boldsymbol{\theta}_t} \|I_{\text{rendered}}(\boldsymbol{\theta}_t) - I_{\text{gt}}\|_1 + \lambda_{\text{SSIM}} \big(1 - \text{SSIM}(I_{\text{rendered}}, I_{\text{gt}})\big) \tag{5.4}$$

Key Idea

렌더링 기반 자세 정제는 2D 관절 검출이 실패하는 관절(예: 오클루전된 팔꿈치)에서도 포토메트릭 정보를 활용하여 자세를 교정할 수 있다는 점에서 강력하다. 단, 학습 초반에 Gaussian이 충분히 수렴하지 않은 상태에서 자세를 함께 최적화하면 국소해에 빠질 수 있으므로, 자세 정제는 사전 학습된 Gaussian 위에서 수행한다.

5.4 학습 모니터링

5.4.1 정량적 지표

학습 과정에서 모니터링해야 할 핵심 지표:

Table 5.1: 3DGS 학습 모니터링 지표

지표	수식	목표값	방향
PSNR	$10\log_{10}(1/\mathrm{MSE})$	$> 30\,\mathrm{dB}$	$\uparrow$
SSIM	구조적 유사도	> 0.95	$\uparrow$
LPIPS	지각적 거리	< 0.05	$\downarrow$
# Gaussians	—	$100\mathrm{K} \sim 500\mathrm{K}$	안정

5.4.2 학습 곡선 해석

- **PSNR이 정체**: 학습률 감소 스케줄 확인, Gaussian densification 주기 조정
- **PSNR 급락**: 자세 파라미터 동시 최적화 시 발생 가능 — 자세 학습률 별도 관리
- **Gaussian 수 폭증**: pruning 임계값 조정, opacity reset 주기 확인

5.5 디버깅 가이드

Table 5.2: 3DGS 인체 재구성 실패 모드와 해결법

실패 모드	원인	해결
유령 사지(ghost limbs)	자세 오차로 정규 공간에서 잘못된 위치에 Gaussian 생성	자세 정제 후 재학습, 시간 정규화 강화
얼룩(floater)	배경의 Gaussian이 인체에 붙음	마스크 기반 배경 분리 강화
흐릿한 텍스처	Gaussian이 너무 크거나 수가 부족	densification 임계값 낮춤
색 번짐(color bleeding)	SH 차수가 높아 과적합	SH 차수를 0–1로 제한하고 점진적 증가

Chapter 6

생체역학 지표 산출

6.1 SMPL에서 해부학적 관절 각도로

SMPL-X의 관절 회전 $\boldsymbol{\theta} \in \mathbb{R}^3$ (축-각도 표현)은 직접적으로 임상적 의미를 갖지 않는다. ISB(International Society of Biomechanics) 권장 분해를 통해 해부학적으로 의미 있는 관절 각도로 변환해야 한다.

6.1.1 ISB 분해

어깨 관절을 예로 들면, 관절 회전 행렬 $\mathbf{R}_{\text{shoulder}}$를 Y-X'-Y'' 오일러 각으로 분해한다:

$$\mathbf{R}_{\text{shoulder}} = \mathbf{R}_y(\alpha)\,\mathbf{R}_{x'}(\beta)\,\mathbf{R}_{y''}(\gamma) \tag{6.1}$$

여기서:

- α: 수평면 거상각(plane of elevation)
- β: 거상각(elevation angle)
- γ: 내/외회전(axial rotation)

Biomechanics

투구 동작에서의 핵심 관절 각도:

- **어깨 외회전(MER)**: 투구 중 최대 외회전 각도, 일반적으로 160–180°
- **팔꿈치 굴곡**: 가속기 시작 시 약 90–100°
- **몸통 회전**: 골반 대비 상체 회전, 최대 40–60° 분리

SMPL-X 관절 회전에서 ISB 각도 추출

```python
from scipy.spatial.transform import Rotation

def smplx_to_isb_shoulder(pose_params, parent_orient):
    """Extract ISB shoulder angles from SMPL-X."""
    # SMPL-X uses axis-angle representation
    R_local = Rotation.from_rotvec(pose_params).as_matrix()
    R_parent = Rotation.from_rotvec(parent_orient).as_matrix()

    # Joint rotation in parent frame
    R_joint = R_parent.T @ R_local

    # YXY Euler decomposition (ISB shoulder)
    r = Rotation.from_matrix(R_joint)
    angles = r.as_euler('YXY', degrees=True)

    return {
        'plane_of_elevation': angles[0],
        'elevation': angles[1],
        'axial_rotation': angles[2]
    }
```

6.2 가상 마커 추출과 OpenSim IK

SMPL-X 메시 표면에 가상 마커를 정의하여 OpenSim의 역운동학(IK)에 입력할 수 있다. 이는 마커리스 모션캡처와 전통적 생체역학 소프트웨어를 연결하는 핵심 단계이다.

가상 마커에서 OpenSim IK까지

1. SMPL-X 메시에서 해부학적 랜드마크에 해당하는 정점(vertex) 인덱스 정의
2. 프레임별 SMPL-X 출력에서 해당 정점의 3D 좌표 추출
3. TRC (Track Row Column) 형식으로 변환
4. OpenSim IK 도구에 입력하여 근골격 모델의 관절 각도 산출

$$\mathbf{p}_{\text{marker}}^{(m)}(t) = \sum_{i \in \mathcal{N}(m)} \alpha_i \, \mathbf{v}_i(t) \tag{6.2}$$

여기서 $\mathcal{N}(m)$은 마커 m 주변의 정점 집합, α_i는 바리센트릭 가중치이다.

6.3 역동역학(Inverse Dynamics)

관절 각도($\mathbf{q}$), 각속도($\dot{\mathbf{q}}$), 각가속도($\ddot{\mathbf{q}}$)와 외력 데이터로부터 관절 모멘트와 힘을 계산한다.

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) - \mathbf{J}^\top \mathbf{F}_{\text{ext}} \tag{6.3}$$

여기서 $\mathbf{M}$은 질량 행렬, $\mathbf{C}$는 코리올리/원심력 항, $\mathbf{G}$는 중력 항, $\mathbf{F}_{\text{ext}}$는 외력(지면반력 등)이다.

Important

마커리스 시스템에서 역동역학을 수행할 때 가장 큰 도전은 정확한 외력 측정이다. Vicon + 지면반력 측정기(force plate)가 있다면 직접 사용하고, 그렇지 않으면 접촉 모델(contact model)이나 잔차 축소 알고리즘(residual reduction)을 적용해야 한다.

6.4 투구 이벤트 자동 검출

투구 동작을 구간별로 분석하기 위해 핵심 이벤트를 자동 검출한다.

Table 6.1: 투구 이벤트 정의

이벤트	정의	검출 기준
FC (Foot Contact)	앞발이 지면에 착지	앞발 수직 속도 ≈ 0, 높이 최저
MER (Max External Rotation)	어깨 최대 외회전	외회전 각도 극대값
BR (Ball Release)	공이 손에서 이탈	손 가속도 급변, 공 위치 분기
MIR (Max Internal Rotation)	어깨 최대 내회전	감속기 내회전 극대값

이벤트 검출 알고리즘

```python
from scipy.signal import argrelextrema

def detect_mer(shoulder_ext_rot, fps=240):
    """Detect Maximum External Rotation event."""
    # Find local maxima in external rotation
    maxima = argrelextrema(shoulder_ext_rot, np.greater,
                           order=int(fps*0.05))[0]
    if len(maxima) == 0:
        return None
    # MER is the global maximum during arm cocking
    mer_idx = maxima[np.argmax(shoulder_ext_rot[maxima])]
    return mer_idx

def detect_foot_contact(foot_height, foot_velocity, fps=240):
    """Detect Foot Contact (stride foot)."""
    # Height below threshold AND velocity near zero
    low = foot_height < np.percentile(foot_height, 10)
    slow = np.abs(foot_velocity) < 0.5  # m/s
    candidates = np.where(low & slow)[0]
    return candidates[0] if len(candidates) > 0 else None
```

6.5 핵심 지표 산출

6.5.1 외반 토크 (Valgus Torque)

팔꿈치 내측측부인대(UCL)에 가해지는 부하의 대리 지표:

$$\tau_{\text{valgus}} = \tau_{\text{elbow,varus/valgus}}\Big|_{t=t_{\text{MER}}} \tag{6.4}$$

프로 투수의 경우 일반적으로 60–120 N · m 범위이며, 이 값이 높을수록 UCL 손상 위험이 증가한다.

6.5.2 최대 외회전 각도

$$\mathrm{MER} = \max_{t \in [t_{\mathrm{FC}}, t_{\mathrm{BR}}]} \theta_{\mathrm{ext_rot}}(t) \tag{6.5}$$

6.5.3 팔 속도

볼 릴리스 시점의 손 선속도:

$$v_{\mathrm{hand}} = \left\| \left. \frac{d\mathbf{p}_{\mathrm{hand}}}{dt} \right|_{t=t_{\mathrm{BR}}} \right\| \tag{6.6}$$

Injury Risk

UCL 부하 기준값:

- $\tau_{\mathrm{valgus}} < 60\ \mathrm{N \cdot m}$: 일반적으로 안전
- $60 \leq \tau_{\mathrm{valgus}} < 90\ \mathrm{N \cdot m}$: 주의 필요 (반복 부하 누적)
- $\tau_{\mathrm{valgus}} \geq 90\ \mathrm{N \cdot m}$: 높은 위험 — 메카닉 교정 권장

이 기준값은 체중, 연령, 투구 유형에 따라 달라지므로 절대적 기준으로 사용해서는 안 된다.

Chapter 7

Vicon 검증과 오차 분석

마커리스 파이프라인의 신뢰성을 확립하기 위해 Vicon gold-standard와의 체계적 비교가 필수적이다.

7.1 시간 정합 확인

두 시스템의 데이터를 비교하기 전에 시간축이 정확히 정합되어야 한다. 크로스-코릴레이션(cross-correlation)으로 최적 시간 오프셋을 찾는다:

$$\Delta t^* = \arg\max_{\Delta t} \sum_t s_{\text{vicon}}(t) \cdot s_{\text{markerless}}(t + \Delta t) \tag{7.1}$$

여기서 s는 빠르게 변하는 신호(예: 손 가속도의 크기)를 사용한다.

크로스-코릴레이션 시간 정합

```python
from scipy.signal import correlate

def find_time_offset(sig1, sig2, fs):
    """Find time offset between two signals."""
    # Normalize
    s1 = (sig1 - sig1.mean()) / sig1.std()
    s2 = (sig2 - sig2.mean()) / sig2.std()

    corr = correlate(s1, s2, mode='full')
    lag = np.arange(-len(s2)+1, len(s1))
    offset_samples = lag[np.argmax(corr)]
    offset_sec = offset_samples / fs
    return offset_sec
```

7.2 자세 정확도 평가

7.2.1 MPJPE (Mean Per Joint Position Error)

가장 기본적인 3D 자세 정확도 지표:

$$\text{MPJPE} = \frac{1}{T \cdot J} \sum_{t=1}^{T} \sum_{j=1}^{J} \left\| \hat{\mathbf{p}}_j(t) - \mathbf{p}_j^{\text{gt}}(t) \right\|_2 \tag{7.2}$$

Procrustes 정합 후의 PA-MPJPE도 함께 보고한다:

$$\text{PA-MPJPE} = \frac{1}{TJ} \sum_{t,j} \left\| s^* \mathbf{R}^* \hat{\mathbf{p}}_j(t) + \mathbf{t}^* - \mathbf{p}_j^{\text{gt}}(t) \right\|_2 \tag{7.3}$$

7.2.2 관절각 RMSE

관절별 각도 오차를 평가한다:

$$\text{RMSE}_\theta = \sqrt{\frac{1}{T} \sum_{t=1}^{T} \left(\theta_{\text{markerless}}(t) - \theta_{\text{vicon}}(t)\right)^2} \tag{7.4}$$

Table 7.1: 기대 정확도 수준 (투구 동작 기준)

지표	우수	수용 가능
MPJPE	$< 20\,\text{mm}$	$< 40\,\text{mm}$
PA-MPJPE	$< 15\,\text{mm}$	$< 30\,\text{mm}$
어깨 각도 RMSE	$< 5^\circ$	$< 10^\circ$
팔꿈치 각도 RMSE	$< 3^\circ$	$< 8^\circ$

7.3 운동역학 정확도

토크와 힘의 비교는 역운동학 오차가 미분으로 증폭되므로 더 엄격한 기준이 필요하다.

$$\text{nRMSE}_\tau = \frac{\text{RMSE}_\tau}{\tau_{\max}^{\text{gt}} - \tau_{\min}^{\text{gt}}} \times 100\,\% \tag{7.5}$$

일반적으로 nRMSE $< 10\%$를 목표로 한다.

7.4 Bland–Altman 분석과 ICC

7.4.1 Bland–Altman 분석

두 측정 방법 간의 일치도를 평가하는 표준 방법이다.

Definition 7.1 (Bland–Altman 플롯)**.** 각 측정에 대해:

- x축: 두 방법의 평균 $\bar{x}_i = (x_i^{(1)} + x_i^{(2)})/2$
- y축: 두 방법의 차이 $d_i = x_i^{(1)} - x_i^{(2)}$

바이어스(bias)는 $\bar{d} = \frac{1}{n}\sum d_i$ 이고, 95% 일치한계(limits of agreement, LoA)는 $\bar{d} \pm 1.96 \cdot s_d$ 이다.

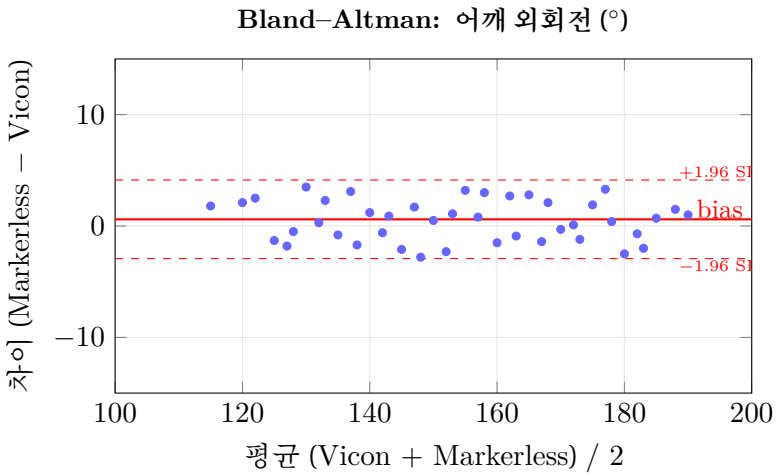


Figure 7.1: Bland–Altman 플롯 예시: 어깨 외회전 각도

7.4.2 ICC (Intraclass Correlation Coefficient)

$$\mathrm{ICC}(3,1) = \frac{\mathrm{MS}_R - \mathrm{MS}_E}{\mathrm{MS}_R + (k-1)\mathrm{MS}_E} \tag{7.6}$$

여기서 MS_R은 피험자 간 평균제곱, MS_E는 잔차 평균제곱, k는 측정 방법 수(=2)이다.

Table 7.2: ICC 해석 기준 [9]

ICC 값	**해석**
< 0.50	Poor
$0.50 \sim 0.75$	Moderate
$0.75 \sim 0.90$	Good
> 0.90	Excellent

7.5 오차 원인 분석

파이프라인의 각 단계에서 발생하는 오차를 분리하여 분석한다.

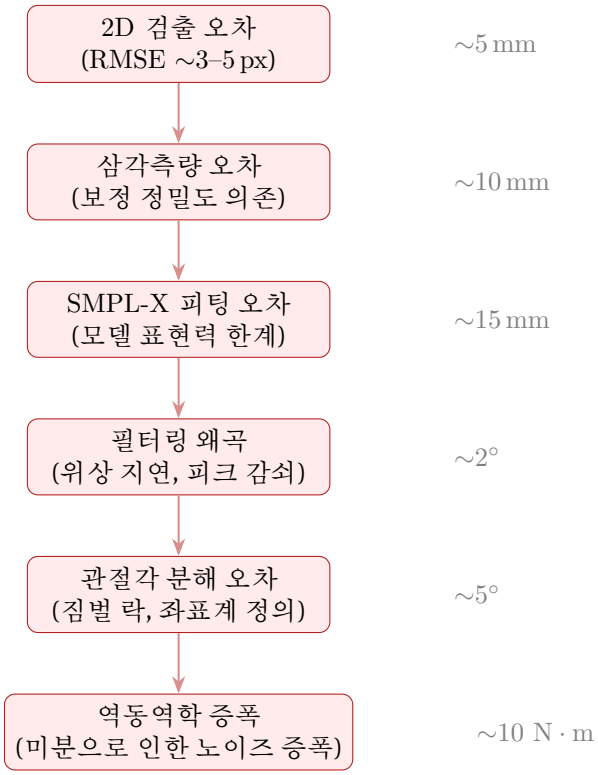


Figure 7.2: 오차 전파 경로와 각 단계의 예상 오차 크기

Key Idea

오차 분석의 핵심 교훈: 역동역학 지표의 정확도는 파이프라인 **최초 단계**(2D 검출, 카메라 보정)의 정밀도에 의해 근본적으로 제한된다. 마지막 단계(역동역학)만 개선해서는 전체 정확도를 높일 수 없다. "쓰레기가 들어가면 쓰레기가 나온다(GIGO)" 원칙이 생체역학 파이프라인에서도 그대로 적용된다.

Chapter 8

시각화와 보고서 생성

8.1 3DGS 렌더링 + 생체역학 오버레이

GART로 재구성한 포토리얼리스틱 인체 위에 생체역학 정보를 시각적으로 오버레이하면, 움직임과 부하를 직관적으로 이해할 수 있다.

8.1.1 관절 토크 컬러맵

각 관절의 토크 크기를 컬러맵으로 매핑하여 메시 위에 표시한다:

$$\text{color}(j, t) = \text{colormap}\left(\frac{|\tau_j(t)| - \tau_{\min}}{\tau_{\max} - \tau_{\min}}\right) \tag{8.1}$$

토크 오버레이 렌더링

```python
import matplotlib.cm as cm

def overlay_torque_on_mesh(vertices, faces, joint_torques,
                           joint_indices, skinning_weights):
    """Map joint torques to vertex colors."""
    # Normalize torques to [0, 1]
    t_norm = (np.abs(joint_torques) - t_min) / (t_max - t_min)
    t_norm = np.clip(t_norm, 0, 1)

    # Map to vertices via skinning weights
    vertex_values = skinning_weights @ t_norm

    # Apply colormap (coolwarm: blue=low, red=high)
    colors = cm.coolwarm(vertex_values)[:, :3]
    return colors
```

8.1.2 벡터 필드 시각화

관절 모멘트의 방향과 크기를 3D 화살표로 표시한다. 특히 팔꿈치 외반 모멘트의 방향은 UCL 부하를 직관적으로 보여준다.

8.2 시계열 관절 각도/토크 플롯

투구 사이클 전체에 걸친 관절 각도와 토크의 시간 변화를 그래프로 나타낸다.

시계열 플롯 생성

```python
import matplotlib.pyplot as plt

fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True)

# Shoulder external rotation
axes[0].plot(time, shoulder_ext_rot, 'b-', label='Markerless')
axes[0].plot(time, vicon_ext_rot, 'r--', label='Vicon')
axes[0].axvline(t_mer, color='gray', ls=':', label='MER')
axes[0].set_ylabel('External Rot. (deg)')
axes[0].legend()

# Elbow flexion
axes[1].plot(time, elbow_flex, 'b-')
axes[1].plot(time, vicon_elbow, 'r--')
axes[1].set_ylabel('Elbow Flex. (deg)')

# Valgus torque
axes[2].plot(time, valgus_torque, 'b-')
axes[2].plot(time, vicon_torque, 'r--')
axes[2].set_ylabel('Valgus Torque (Nm)')
axes[2].set_xlabel('Time (s)')

plt.tight_layout()
plt.savefig('pitching_timeseries.pdf', dpi=300)
```

8.3 투구별 비교 대시보드

여러 투구를 시간 정규화(time-normalized)하여 중첩 비교한다. 투구 사이클은 FC→MER→BR→MIR의 4개 이벤트로 구분하고, 각 구간을 0–100%로 정규화한다.

Table 8.1: 투구별 비교 대시보드 항목

항목	내용
3D 시각화	4개 이벤트 시점의 3DGS 렌더링 스냅샷
각도 프로파일	어깨/팔꿈치 각도의 시간 정규화 곡선 (평균±SD 밴드)
토크 프로파일	팔꿈치 외반 토크의 시간 정규화 곡선
핵심 지표 표	MER, 외반 토크, 팔 속도, 보폭 등 스칼라 지표

8.4 자동 보고서 생성 파이프라인

분석 결과를 자동으로 PDF 보고서로 생성하는 파이프라인을 구축한다.

Jinja2 기반 LaTeX 보고서 자동 생성

```python
from jinja2 import Environment, FileSystemLoader

env = Environment(loader=FileSystemLoader('templates/'))
template = env.get_template('pitching_report.tex.j2')

report = template.render(
    subject_id='S01',
    date='2026-03-15',
    pitches=pitch_results,  # list of per-pitch metrics
    summary_stats=summary,  # aggregate statistics
    figure_paths=fig_paths  # generated plot paths
)

with open('reports/S01_report.tex', 'w') as f:
    f.write(report)

# Compile
os.system('xelatex reports/S01_report.tex')
```

Chapter 9

수학적 부록

9.1 Procrustes 정합

9.1.1 문제 정의

N 개의 대응 점 쌍 $\{(\mathbf{a}_i, \mathbf{b}_i)\}_{i=1}^{N}$ 이 주어졌을 때, 최적의 강체 변환(회전 $\mathbf{R}$, 이동 $\mathbf{t}$)과 스케일 s를 구한다:

$$\min_{\mathbf{R}\in SO(3),\,\mathbf{t},\,s} \sum_{i=1}^{N} \|\mathbf{b}_i - (s\mathbf{R}\mathbf{a}_i + \mathbf{t})\|^2 \tag{9.1}$$

9.1.2 풀이

Step 1. 중심 이동:

$$\bar{\mathbf{a}} = \frac{1}{N}\sum_i \mathbf{a}_i, \quad \bar{\mathbf{b}} = \frac{1}{N}\sum_i \mathbf{b}_i, \quad \tilde{\mathbf{a}}_i = \mathbf{a}_i - \bar{\mathbf{a}}, \quad \tilde{\mathbf{b}}_i = \mathbf{b}_i - \bar{\mathbf{b}} \tag{9.2}$$

Step 2. 교차공분산 행렬과 SVD:

$$\mathbf{H} = \sum_{i=1}^{N} \tilde{\mathbf{a}}_i \tilde{\mathbf{b}}_i^\top, \quad \mathbf{H} = \mathbf{U}\mathbf{\Lambda}\mathbf{V}^\top \tag{9.3}$$

Step 3. 최적 회전:

$$\mathbf{R}^* = \mathbf{V}\begin{pmatrix} 1 & & \\ & 1 & \\ & & \det(\mathbf{V}\mathbf{U}^\top) \end{pmatrix}\mathbf{U}^\top \tag{9.4}$$

Step 4. 최적 스케일과 이동:

$$s^* = \frac{\operatorname{tr}(\mathbf{\Lambda}\cdot\operatorname{diag}(1,1,\det(\mathbf{V}\mathbf{U}^\top)))}{\sum_i \|\tilde{\mathbf{a}}_i\|^2}, \quad \mathbf{t}^* = \bar{\mathbf{b}} - s^*\mathbf{R}^*\bar{\mathbf{a}} \tag{9.5}$$

9.2 Butterworth 필터 설계

9.2.1 아날로그 프로토타입

n 차 Butterworth 저역통과 필터의 전달 함수:

$$H(s) = \frac{1}{\prod_{k=1}^{n}(s - p_k)}, \quad p_k = e^{j\pi(2k+n-1)/(2n)}, \quad k = 1, \ldots, n \tag{9.6}$$

9.2.2 디지털 변환

쌍일차 변환(bilinear transform)으로 아날로그 필터를 디지털로 변환한다:

$$s = \frac{2}{T_s} \cdot \frac{1 - z^{-1}}{1 + z^{-1}} \tag{9.7}$$

사전 왜곡(pre-warping)을 적용하여 차단 주파수의 정확성을 보장한다:

$$\Omega_c = \frac{2}{T_s} \tan\left(\frac{\omega_c T_s}{2}\right) \tag{9.8}$$

9.2.3 양방향 필터링(Zero-Phase)

`filtfilt`은 필터를 순방향·역방향으로 두 번 적용하여 위상 지연을 제거한다:

$$|H_{\text{zp}}(f)|^2 = |H(f)|^4 = \frac{1}{(1 + (f/f_c)^{2n})^2} \tag{9.9}$$

유효 차수가 $2n$이 되므로, 실제 차단 특성은 단방향 대비 두 배 급격해진다.

9.3 Bland–Altman 분석의 통계적 기초

9.3.1 가정

- 차이 d_i가 정규분포를 따른다: $d_i \sim \mathcal{N}(\mu_d, \sigma_d^2)$
- 차이의 크기가 측정값의 크기에 의존하지 않는다 (비례 바이어스 없음)
- 관측치가 서로 독립이다

9.3.2 일치한계(Limits of Agreement)

$$\text{LoA} = \bar{d} \pm 1.96 \cdot s_d \tag{9.10}$$

여기서 $\bar{d} = \frac{1}{n}\sum_{i=1}^{n} d_i$, $s_d = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(d_i - \bar{d})^2}$이다.

95% 신뢰구간을 포함한 LoA:

$$\text{SE(LoA)} = \sqrt{\frac{1}{n} + \frac{1.96^2}{2(n-1)}} \cdot s_d \tag{9.11}$$

9.3.3 비례 바이어스 검정

차이 d_i를 평균 $\bar{x}_i$에 대해 회귀 분석하여 기울기가 0인지 검정한다:

$$d_i = \beta_0 + \beta_1 \bar{x}_i + \epsilon_i \tag{9.12}$$

β_1이 유의하면 비례 바이어스가 존재하며, 이 경우 로그 변환 후 재분석하거나 회귀 기반 LoA를 사용한다.

9.4 ICC 계산

9.4.1 Two-Way Mixed Model, Absolute Agreement

ICC(3,1)의 분산분석 모형:

$$x_{ij} = \mu + r_i + c_j + e_{ij} \tag{9.13}$$

여기서 r_i는 피험자 효과(random), c_j는 방법 효과(fixed), e_{ij}는 잔차이다.

$$\mathrm{ICC}(3,1) = \frac{\mathrm{MS}_R - \mathrm{MS}_E}{\mathrm{MS}_R + (k-1)\mathrm{MS}_E} \tag{9.14}$$

9.4.2 신뢰구간

ICC의 95% 신뢰구간은 F-분포를 이용하여 계산한다:

$$\begin{aligned} F_L &= \frac{F_0}{F_{1-\alpha/2,n-1,(n-1)(k-1)}}, \quad F_U = F_0 \cdot F_{1-\alpha/2,(n-1)(k-1),n-1} \\ \mathrm{ICC}_L &= \frac{F_L - 1}{F_L + k - 1}, \quad \mathrm{ICC}_U = \frac{F_U - 1}{F_U + k - 1} \end{aligned} \tag{9.15}$$

여기서 $F_0 = \mathrm{MS}_R/\mathrm{MS}_E$이다.

ICC 계산 구현

```python
import pingouin as pg
import pandas as pd

def compute_icc(vicon_data, markerless_data, metric_name):
    """Compute ICC(3,1) for a single metric."""
    n = len(vicon_data)
    df = pd.DataFrame({
        'subject': list(range(n)) * 2,
        'method': ['vicon']*n + ['markerless']*n,
        'value': np.concatenate([vicon_data, markerless_data])
    })

    icc = pg.intraclass_corr(
        data=df, targets='subject',
        raters='method', ratings='value'
    )
    # ICC3 = two-way mixed, single measures
    result = icc.set_index('Type').loc['ICC3']
    return result['ICC'], result['CI95%']
```

Bibliography

[1] Winter, D.A. (2009). *Biomechanics and Motor Control of Human Movement*. 4th ed. Wiley.

[2] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Trans. PAMI*, 22(11), 1330–1334.

[3] Xu, Y., Zhang, J., Zhang, Q., & Tao, D. (2022). ViTPose: Simple Vision Transformer Baselines for Human Pose Estimation. *NeurIPS*.

[4] Jiang, T., Lu, P., Zhang, L., et al. (2023). RTMPose: Real-Time Multi-Person Pose Estimation based on MMPose. *arXiv:2303.07399*.

[5] Dong, Z., Song, J., Chen, X., et al. (2021). EasyMocap: Open source toolbox for markerless multi-view motion capture.

[6] Pavlakos, G., Choutas, V., Ghorbani, N., et al. (2019). Expressive body capture: 3D hands, face, and body from a single image. *CVPR*.

[7] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM TOG* (*SIGGRAPH*).

[8] Lei, J., Wang, Y., Pavlakos, G., & Daniilidis, K. (2024). GART: Gaussian Articulated Template Models. *CVPR*.

[9] Koo, T.K. & Li, M.Y. (2016). A guideline of selecting and reporting intraclass correlation coefficients for reliability research. *J Chiropr Med*, 15(2), 155–163.

[10] Fleisig, G.S., Barrentine, S.W., Zheng, N., et al. (2011). Kinetic comparison among the fastball, curveball, change-up, and slider in collegiate baseball pitchers. *Am J Sports Med*.

[11] Schönberger, J.L. & Frahm, J.-M. (2016). Structure-from-Motion Revisited. *CVPR*.

[12] Bland, J.M. & Altman, D.G. (1986). Statistical methods for assessing agreement between two methods of clinical measurement. *The Lancet*, 327(8476), 307–310.

[13] Wu, G., van der Helm, F.C.T., et al. (2005). ISB recommendation on definitions of joint coordinate systems. *J Biomech*, 38(5), 981–992.

[14] Loper, M., Mahmood, N., Romero, J., Pons-Moll, G., & Black, M.J. (2015). SMPL: A skinned multi-person linear model. *ACM TOG* (*SIGGRAPH Asia*).

[15] Pavlakos, G., Choutas, V., Ghorbani, N., et al. (2019). Expressive body capture: 3D hands, face, and body from a single image. *CVPR*.

용어집

용어	설명
3DGS	3D Gaussian Splatting. 3D 공간의 Gaussian 원소를 래스터화하여 장면을 렌더링하는 기법
BA	Bundle Adjustment. 카메라 자세와 3D 점 위치를 동시에 최적화하는 비선형 최소제곱
Bland–Altman	두 측정 방법 간의 일치도를 평가하는 통계적 방법
COLMAP	Structure from Motion과 Multi-View Stereo를 수행하는 오픈소스 도구
DLT	Direct Linear Transform. 다시점 삼각측량의 선형 해법
FC	Foot Contact. 투구 시 앞발의 지면 접촉 이벤트
GART	Gaussian Articulated Representation of Transformable humans
ICC	Intraclass Correlation Coefficient. 측정 신뢰도 지표
ID	Inverse Dynamics. 운동학 데이터로부터 관절 힘/모멘트를 역으로 계산
IK	Inverse Kinematics. 관절 각도를 역으로 계산하는 방법
ISB	International Society of Biomechanics. 관절 좌표계의 표준을 정의
LBS	Linear Blend Skinning. 스켈레톤 기반 메시 변형 기법
LoA	Limits of Agreement. Bland–Altman 분석의 95% 일치한계
LPIPS	Learned Perceptual Image Patch Similarity. 지각적 이미지 유사도 지표
MER	Maximum External Rotation. 투구 중 어깨의 최대 외회전
MIR	Maximum Internal Rotation. 투구 중 어깨의 최대 내회전
MPJPE	Mean Per Joint Position Error. 3D 자세 추정 오차 지표
MVS	Multi-View Stereo. 다시점 이미지에서 밀집 3D 복원
PA-MPJPE	Procrustes Aligned MPJPE. 최적 강체 정합 후의 관절 위치 오차
PSNR	Peak Signal-to-Noise Ratio. 이미지 품질 지표
RANSAC	Random Sample Consensus. 이상치에 강건한 모델 추정
SfM	Structure from Motion. 다시점 이미지에서 카메라 자세와 희소 3D 복원
SH	Spherical Harmonics. 방향 의존적 색상을 표현하는 기저 함수
SMPL-X	Skinned Multi-Person Linear model eXpressive. 손과 얼굴을 포함하는 파라메트릭 인체 모델
SSIM	Structural Similarity Index. 구조적 이미지 유사도 지표
UCL	Ulnar Collateral Ligament. 팔꿈치 내측측부인대

용어	설명
VPoser	Variational Human Pose Prior. 학습된 자세 사전 확률 모델